

ROB 5014 Math and Simulation for Robotics

Lecture: Game Physics Solver

Hyungpil Moon, Professor



Department of Mechanical Engineering
Department of Intelligent Robotics

- Isaac Sim solver: PGS
 - Projected Gauss–Seidel method
 - E. Catto, “Iterative Dynamics with Temporal Coherence,” presented at the Game Developers Conference, 2005.
- PGS is only for gaming physics, not for real robotics because of ignorance of inertia terms, complex frictional contacts, etc.

- Linear velocity: v
- Angular velocity: ω
- Kinematic differential equation for a rigid body

- $\dot{x} = v$

- $\dot{q} = \frac{1}{2} \omega * q \cdots (1)$

- For a system of n bodies

- $V = \begin{pmatrix} v_1 \\ \omega_1 \\ \vdots \\ v_n \\ \omega_n \end{pmatrix}$: $6n \times 1$ column vector

- Proof of (1)

- Quaternion: q

- Two consecutive rotations: pq

- Rotate by q and then p (both quaternion)

- A body rotates at a constant angular velocity $\omega(t)$ for Δt

- $q(t_0 + \Delta t) = \left[\cos \frac{|\omega(t_0)|\Delta t}{2}, \sin \frac{|\omega(t_0)|\Delta t}{2} \hat{\omega}(t_0) \right] q(t_0)$

- Let $t = t_0 + \Delta t$

- $q(t) = \left[\cos \frac{|\omega(t_0)|(t-t_0)}{2}, \sin \frac{|\omega(t_0)|(t-t_0)}{2} \hat{\omega}(t_0) \right] q(t_0)$

- We want to differentiate the above

- Note that $q(t_0)$ is constant

- At $t = t_0$,

- $\frac{d}{dt} \left(\cos \frac{|\omega(t_0)|(t-t_0)}{2} \right) = 0, \frac{d}{dt} \left(\sin \frac{|\omega(t_0)|(t-t_0)}{2} \right) = \frac{|\omega(t_0)|}{2}$

- Thus, at $t = t_0$, $\dot{q}(t) = \left[0, \frac{|\omega(t_0)|}{2} \hat{\omega}(t_0) \right] q(t_0) = \frac{1}{2} [0, \omega(t_0)] q(t_0) = \frac{1}{2} \omega(t) * q(t)$

- Constraints between rigid bodies
 - Joints, contact, etc.
- Position constraints: C_k for body i, j whose position x and orientation q
 - $C_k(x_i, q_i, x_j, q_j) = 0$: $s \times 1$ column vector, s : number of constraints
- Note that
 - $\dot{C} = JV = 0$
 - Admissible velocity is orthogonal to the rows of J
- Constraint forces
 - $F_c = J^T \lambda$
 - λ : constraint force
 - Constraint forces do no work, thus they must be orthogonal to V

- Computing the Jacobian
- Constraint: distance between the rigid bodies

- $C_{dist} = \frac{1}{2} [(p_2 - p_1)^2 - L^2]$

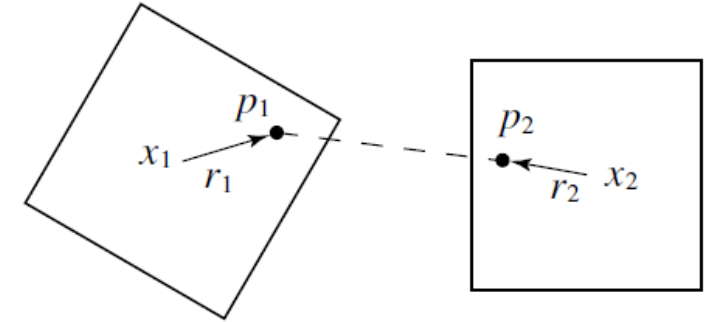
- Jacobian

- $\dot{C}_{dist} = d \cdot (v_2 + \omega_2 \times r_2 - v_1 - \omega_1 \times r_1)$

- $d = p_2 - p_1$

- Using $A \cdot B \times C = B \cdot C \times A$

- $\dot{C}_{dist} = (-d^T \quad -(r_1 \times d)^T \quad d^T \quad (r_2 \times d)^T) \begin{pmatrix} v_1 \\ \omega_1 \\ v_2 \\ \omega_2 \end{pmatrix} = JV$



$$p(t) = R(t)p_0 + x(t)$$

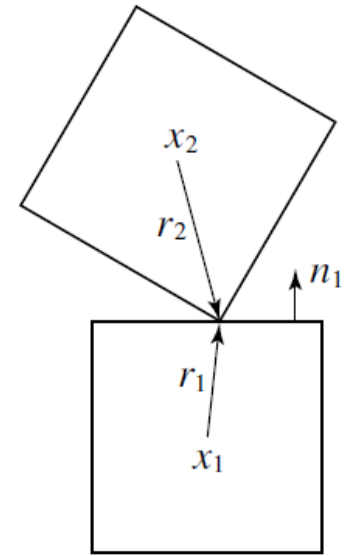
$$r(t) = p(t) - x(t)$$

$$\begin{aligned} \dot{p}(t) &= \dot{R}(t)p_0 + \dot{x}(t) = \omega(t)^* R(t)p_0 + v(t) \\ &= \omega(t) \times (R(t)p_0 + x(t) - x(t)) + v(t) \\ &= \omega(t) \times (p(t) - x(t)) + v(t) \\ &= \omega(t) \times r(t) + v(t) \end{aligned}$$

- Normal Constraint

- Position constraint: $C_n = (x_2 + r_2 - x_1 - r_1) \cdot n_1 < 0$ when overlapping
- Velocity constraint:
 - $\dot{C}_n = (v_2 + \omega_2 \times r_2 - v_1 - \omega_1 \times r_1) \cdot n_1 + \underbrace{(x_2 + r_2 - x_1 - r_1) \cdot \omega_1 \times n_1}_{\text{If penetration is small, this is ignored}}$

If penetration is small, this is ignored



- Simplified Jacobian

- $J_n V = \begin{pmatrix} -n^T & -(r_1 \times n)^T & n^T & (r_2 \times n)^T \end{pmatrix} \begin{pmatrix} v_1 \\ \omega_1 \\ v_2 \\ \omega_2 \end{pmatrix}$

- Constraint force $0 \leq \lambda_n < \infty$: the normal force can push to the bodies, but not pull them together

- Penetration handling
 - When overlapped, the bodies should be pushed apart: $J_n V = -\beta C_n$
 - β : the speed of penetration resolution
 - $C_n = (x_2 + r_2 - x_1 - r_1) \cdot n_1$
 - Recall that $\dot{C}_n = J_n V$, thus $\dot{C}_n + \beta C_n = 0 \rightarrow C_n(t) = e^{-\beta t} C_n(0)$
- The choice of β by a simple example
 - A rigid body whose center of mass (x) constrained to the point p
 - Constraint equation: $C = x - p$
 - Body velocity: $v + \beta(x - p) = 0 \rightarrow v = -\beta(x - p)$
 - Using Euler equation: $x(t + \Delta t) = x(t) + \Delta t v(t) = x(t) - \Delta t \beta(x(t) - p)$
 - Thus, $x(t + \Delta t) - p = (1 - \Delta t \beta)(x(t) - p) \rightarrow C(k\Delta t) = (1 - \Delta t \beta)^k C(0)$
 - Clearly, x will converse to p iff $0 < \beta < \frac{2}{\Delta t}$
 - Moreover, $0 < \beta < \frac{1}{\Delta t}$, then the position error will decay smoothly to zero,
 - For $\frac{1}{\Delta t} < \beta < \frac{2}{\Delta t}$, the position error will decay with an oscillation.

- Friction Constraints

- For contact normal n , two tangent unit vectors u_1, u_2 such that $u_1 \times u_2 = n$
- Friction constraints: $\dot{C}_{u_i} = (v_2 + \omega_2 \times r_2 - v_1 - \omega_1 \times r_1) \cdot u_i$
- Jacobian: $J_u V = \begin{pmatrix} -u_1^T & -(r_1 \times u_1)^T & u_1^T & (r_2 \times u_1)^T \\ -u_2^T & -(r_1 \times u_2)^T & u_2^T & (r_2 \times u_2)^T \end{pmatrix} \begin{pmatrix} v_1 \\ \omega_1 \\ v_2 \\ \omega_2 \end{pmatrix}$
- Simple bounded force constraints: $-\mu m_c g \leq \lambda_{u_i} \leq \mu m_c g$
- This is only for gaming simulators.

- Newton–Euler equations of motion
 - $m\dot{v} = f_c + f_{ext}$
 - $I\dot{\omega} = \tau_c + \tau_{ext}$
 - f_c, τ_c : constraint force/torque
 - f_{ext}, τ_{ext} : external force/torque
 - This is for gaming simulator only because of neglecting of inertial torque $\omega \times I\omega$.
 - Notice that the robotics arm inertia varies as it changes the configuration.
- $M\dot{V} = J^T \lambda + F_{ext}$
- $JV = \zeta$

- $\dot{V} \approx \frac{V^2 - V^1}{\Delta t}$ (the superscript denotes the time step)
- Thus, from $M\dot{V} = J^T \lambda + F_{ext}$, we have $M(V^2 - V^1) = \Delta t(J^T \lambda + F_{ext})$
- Once λ (constraint force) is computed, $M(V^2 - V^1) = \Delta t(J^T \lambda + F_{ext})$ computes V^2 , then the position is updated.
 - $x^2 = x^1 + \Delta t v^2$
 - $q^2 = q^1 + \frac{\Delta t}{2} \omega^2 * q^1$ (the superscript denotes the time step)

- $\dot{V} \approx \frac{V^2 - V^1}{\Delta t}$ (the superscript denotes the time step)
- Thus, from $M\dot{V} = J^T \lambda + F_{ext}$, we have $M(V^2 - V^1) = \Delta t(J^T \lambda + F_{ext})$
- Using $JV^2 = \zeta$, we solve for V^2 in terms of λ to get
- $JM^{-1}J^T \lambda = \frac{1}{\Delta t} \zeta - J(\frac{1}{\Delta t} V^1 + M^{-1}F_{ext})$
- $JB\lambda = \eta$, $B = M^{-1}J^T$, $\eta = \frac{1}{\Delta t} \zeta - J(\frac{1}{\Delta t} V^1 + M^{-1}F_{ext})$
- Solving dynamic becomes a linear equation $Ax = b$ ($A = JB$, $x = \lambda$, $b = \eta$) !!
- Solving $Ax=b$ is important !!! (→ an example method is Gauss-Seidel)
- Dynamics now becomes a Mixed Linear Complementarity Problem (MLCP)
 - $w = JB\lambda - \eta$
 - $\lambda^- \leq \lambda \leq \lambda^+$
 - $w_i = 0 \leftrightarrow \lambda_i^- \leq \lambda_i \leq \lambda_i^+, \forall i$
 - $\lambda_i = \lambda^- \leftrightarrow w_i \geq 0, \forall i$
 - $\lambda_i = \lambda^+ \leftrightarrow w_i \leq 0, \forall i$

Thank you